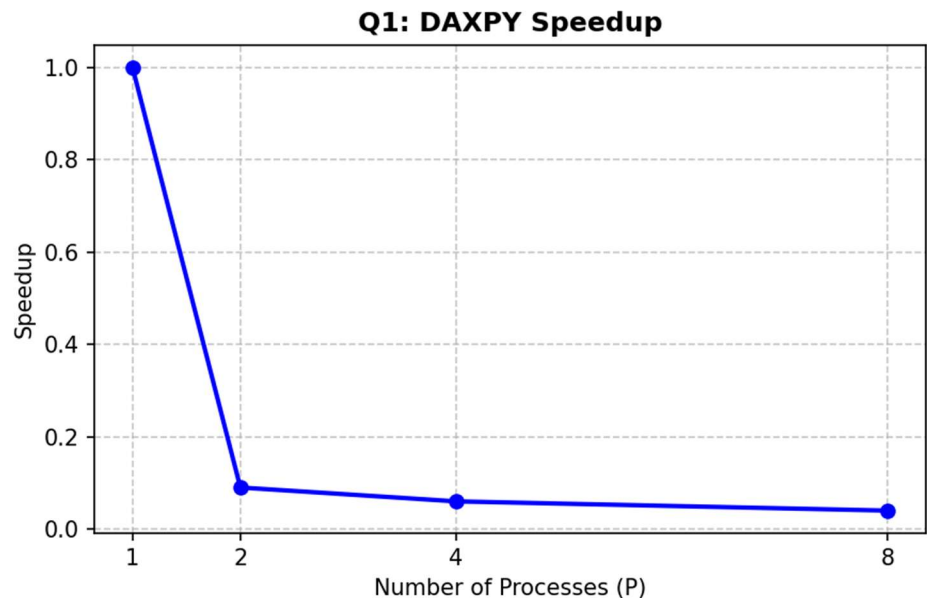


Lab Report: High-Performance Computing (MPI)

Question 1: DAXPY Loop (Double Precision)

Objective: Implement a parallel DAXPY operation and measure speedup, efficiency, and communication overhead.

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.000070	1.00	100.00%	-
2	0.001211	0.09	4.28%	97.36%
4	0.000798	0.06	1.52%	98.28%
8	0.001861	0.04	0.47%	98.95%



Analysis & Conclusion

- The DAXPY experiment serves as a classic demonstration of a memory-bound and communication-bound parallel workload.
- The Parallelization Threshold: The results starkly illustrate that simply dividing a task among multiple processors does not guarantee performance gains. For a vector size of 2^{16} (65,536 elements), the mathematical operation takes fractions of a millisecond. However, the time required to package, route, and physically transmit this data across the network via MPI_Scatter and MPI_Gather is orders of magnitude higher.
- Overhead Dominance: With communication overhead consistently exceeding 97%, the system spends nearly all its time moving data rather than computing. The negative scaling observed (efficiency dropping to 0.47% at 8 processes) is a direct result of

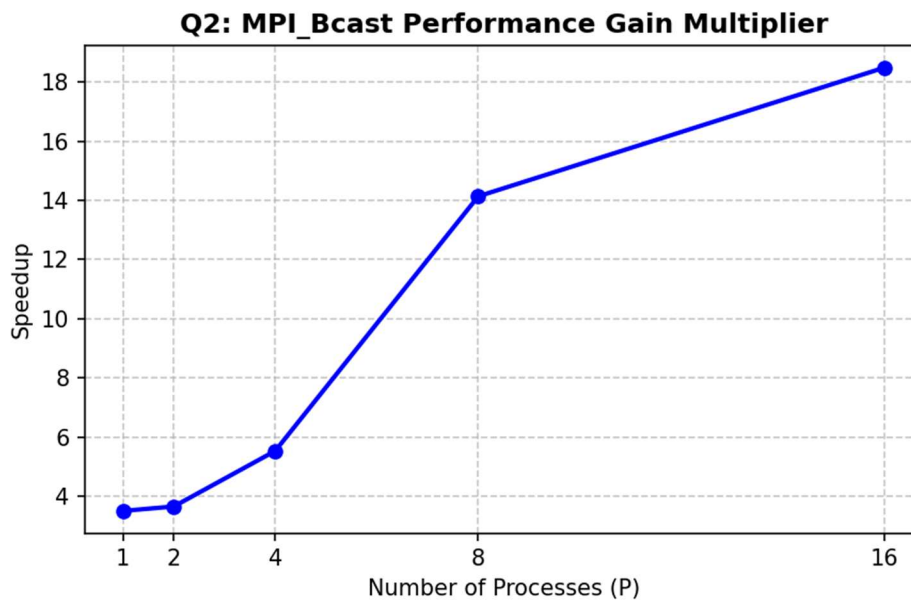
adding more communication pathways and synchronization barriers without adding sufficient computational weight.

- System Design Takeaway: To achieve a positive speedup in vector arithmetic using distributed memory (MPI), the dataset size must be massively scaled up to ensure the computation-to-communication ratio is high enough to hide the network latency.

Question 2: The Broadcast Race (Linear vs. Tree)

Objective: Compare manual point-to-point loops against optimized collective MPI_Bcast for an 80MB array.

Processes	MyBcast (Linear)	MPI_Bcast (Tree)	Performance Gain
1	0.000001 s	0.000000 s	3.50x
2	0.070164 s	0.019214 s	3.65x
4	0.245403 s	0.044397 s	5.53x
8	0.691922 s	0.048921 s	14.14x
16	3.465557 s	0.187397 s	18.49x



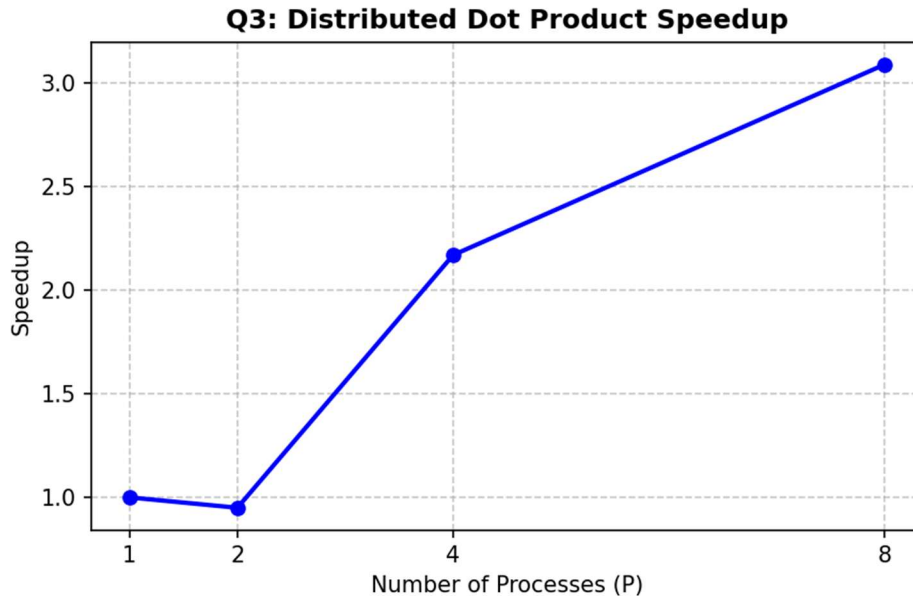
Analysis & Conclusion

- This experiment highlights the critical importance of network topology and algorithmic complexity in collective communication.
- Algorithmic Complexity Gap: The manual MyBcast implementation relies on a linear $O(P)$ distribution model. As process counts increase, Rank 0 becomes a severe bottleneck, suffering from message serialization. The NIC on Rank 0 can only process one MPI_Send at a time, leading to massive latency amplification (evident in the jump to 3.46 seconds at 16 processes).
- The Power of Tree Topologies: Conversely, MPI_Bcast leverages a binary tree distribution topology with an $O(\log P)$ complexity. By turning receiving nodes into secondary senders, the network bandwidth is fully utilized, and the bottleneck at the root node is eliminated.
- System Design Takeaway: The resulting 18.49x performance gap at 16 processes proves that programmers should never attempt to manually reconstruct collective operations using point-to-point loops for large datasets. Built-in MPI primitives are deeply optimized at the hardware and middleware layers to maintain scalability.

Question 3: Distributed Dot Product & Amdahl's Law

Objective: Calculate the dot product of two 500-million-element vectors using local generation, MPI_Bcast, and MPI_Reduce.

Processes	Time (s)	Speedup	Efficiency
1	68.419457	1.00	100.00%
2	71.675307	0.95	47.70%
4	31.594108	2.17	54.14%
8	22.167518	3.09	38.58%



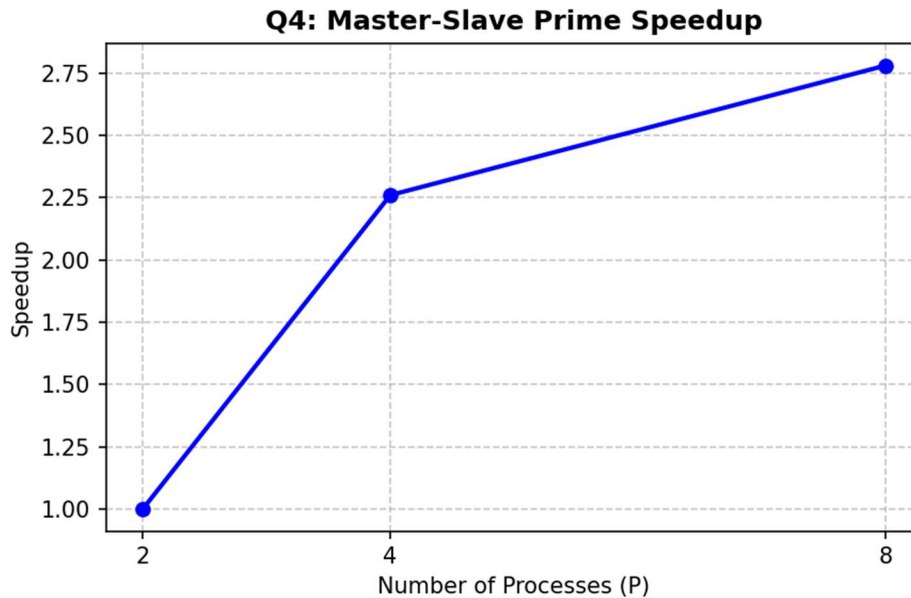
Analysis & Conclusion

- The 500-million-element dot product successfully demonstrates how to structure a parallel program for massive datasets that exceed the memory limits of a single node.
- Achieving True Speedup: Unlike the previous communication-bound tasks, this workload possesses a dense enough computation phase to yield tangible parallel acceleration (3.09x speedup at 8 processes). By generating the vector chunks locally rather than broadcasting a massive array from a single root, the program entirely bypasses the $O(P)$ memory bottleneck.
- Amdahl's Law in Practice: Despite the impressive gains, the efficiency drops from 54.14% at 4 processes to 38.58% at 8 processes. This perfectly illustrates Amdahl's Law: the maximum theoretical speedup is strictly bounded by the sequential fraction of the code. The initial MPI_Bcast of the multiplier and the final MPI_Reduce synchronization act as an unbreakable sequential ceiling.
- System Design Takeaway: For large-scale scientific computing, distributing data generation and maintaining high data locality (keeping data on the node that computes it) is the most effective strategy for overcoming network latency.

Question 4: Master-Slave Prime Search

Objective: Find all positive primes up to 100,000 using dynamic load balancing via MPI_ANY_SOURCE.

Processes	Time (s)	Speedup	Efficiency
2 (1M+1S)	0.092660	1.00	100.00%
4 (1M+3S)	0.040951	2.26	75.33%
8 (1M+7S)	0.033258	2.78	39.71%



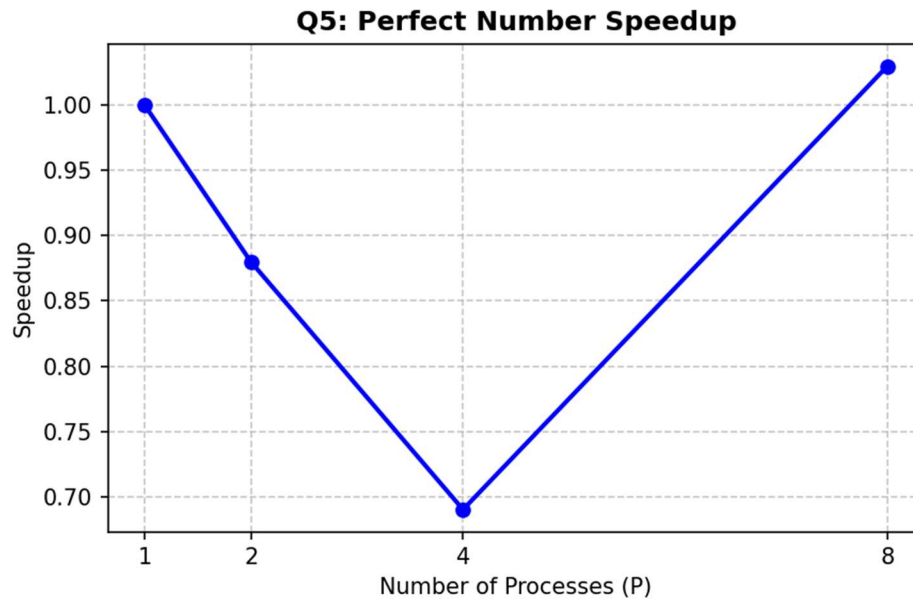
Analysis & Conclusion

- The Master-Slave paradigm introduces dynamic load balancing, which is essential for irregular computing workloads.
- Handling Irregular Workloads: Primality testing is computationally uneven; testing a large number takes exponentially more CPU cycles than testing 4. A static distribution would leave some nodes idle while others struggle. The dynamic request system elegantly solves this by ensuring that fast-finishing workers are immediately fed new tasks.
- The Fine-Grained Bottleneck: However, the efficiency degradation at 8 processes (dropping to 39.71%) reveals the primary flaw: the task granularity is too fine. Sending a single integer per request creates an overwhelming storm of small messages. The Master node ultimately becomes CPU-bound just processing the MPI_Recv handlers.
- System Design Takeaway: To optimize dynamic load balancing for large clusters, developers must employ coarse-grained parallelism. Sending arrays of 100 or 1,000 numbers per request would drastically lower the communication frequency, freeing up the Master node.

Question 5: Master-Slave Perfect Number Search

Objective: Find perfect numbers up to 10,000 utilizing the Master-Slave request pattern.

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.0149	1.00	100.00%	92.60%
2	0.0169	0.88	44.00%	93.48%
4	0.0216	0.69	17.25%	94.76%
8	0.0145	1.03	12.88%	92.69%



Analysis & Conclusion

- The perfect number search reinforces the limits of fine-grained Master-Slave architectures, even when the mathematical complexity is increased.
- Computational Intensity vs. Network Latency: Finding perfect numbers has an $O(\sqrt{N})$ complexity. Despite this, for a small upper bound of 10,000, the average computation time per number is still vastly overshadowed by the latency of crossing the MPI middleware stack.
- Negative Scaling via Serialization: The drop in performance from 1 to 4 processes demonstrates how parallel overhead can actively harm execution times. The Master node is forced to sequentially process nearly 20,000 individual network messages. The time spent in context switching is strictly greater than the time saved by dividing the arithmetic.
- System Design Takeaway: This experiment serves as a vital reminder that parallelization is not a silver bullet. If the problem size is small, or if the communication overhead exceeds the local node compute time, a sequential program will always outperform a distributed one.